complete-http

Based off *Factoring "short-sleeve" RSA keys with polynomials* by Keegan Ryan (Trail of Bits)

yes, this challenge is mostly vibecoded. I don't know C#/dotnet :(

this file and appendix.py are fully human written though :)

intended solution

1. open pcap in wireshark
2. get n from the Certificate TLS record (packet #9)
3. crack the key using a script similar to solve.py
4. load key into wireshark
5. flag is in packet 21 (HTTP response)

responses to the original article appendix: n_2

We know that n_2 is generated with the CompleteFTP RNG as follows:

```
public void genRandomBits(int bits) {
  // Calculate the number of limbs
  int numLimbs = bits / 32;
  // Allocate space for the RNG output
  byte[] array = new byte[numLimbs];
  // Call the system RNG
  rngProvider.GetNonZeroBytes(array);
  // Copy to the limbs of the big number
  Array.Copy(array, 0, bignumLimbs, 0, numLimbs);
  // Set the top bit to ensure proper bit length
  bignumLimbs[numLimbs - 1] |= 0x80000000;
  // Store the length
  dataLength = numLimbs;
}
```

As discussed in the (amazing) original article, the vulnerability lies in the fact that there is "a mismatch between the size of the limbs and the size of the RNG output".

1. If you compute $f_{n_2}(x)$ using $B = 2^{32}$, some of the coefficients are large. Why is that? Is it true that all of the coefficients of $f_p(x)$ and $f_q(x)$ are small?

Consider the coefficients of $f_p(x)$ and $f_q(x)$, i.e. the 32-bit limbs of p and q. From the definition of `genRandomBits()`, we know that each limb is actually a zero-extended 8-bit value, hence $x \in [1, 255] \cap \mathbb{Z}$ ***except the final limb***. The final limb (the most significant, assuming little-endian) has its top bit set. So the result of `genRandomBits(32 * 4)` would look like `[0xab, 0xcd, 0xef, 0x80000012]`.

Therefore it is not true that all of the coefficients of $f_p(x)$ and $f_q(x)$ are small: we know that the last coefficient of both will be the full 32 bits. Because this is the

Let N_i be the i-th 32-bit limb of n_2. By polynomial multiplication:

$$N_i = \sum_{j=0}^{i} p_j q_{i-j}$$

This implies that $\forall i \geq 31$, N_i must contain multiples of the final limb of p and q. So we can expect N_i to be more than 32-bit for $i \geq 31$. In practice due to carrying the values may be less than 32-bit, as we can observe from the limbs as shown below:

```
x=000029e5 bit_length=14
x=0000acdf bit_length=16
x=00010107 bit_length=17
x=000113f4 bit_length=17
x=000151be bit_length=17
x=0001a8a9 bit_length=17
x=0001cdf4 bit_length=17
x=00024384 bit_length=18
x=0002d5a6 bit_length=18
x=0002cbb3 bit_length=18
x=0002de11 bit_length=18
x=0003ada6 bit_length=18
x=0003fe6d bit_length=18
x=0004c564 bit_length=19
x=0004cfbe bit_length=19
x=000457dc bit_length=19
x=00040bf1 bit_length=19
x=000574dd bit_length=19
x=0005da92 bit_length=19
x=000566cc bit_length=19
x=0005ca4c bit_length=19
x=00060593 bit_length=19
x=00062231 bit_length=19
x=0006ec66 bit_length=19
x=0007c0ec bit_length=19
x=000724f1 bit_length=19
x=00072e59 bit_length=19
x=00080db8 bit_length=20
x=0007e843 bit_length=19
x=0009578f bit_length=20
x=0009a6d9 bit_length=20
x=00099a26 bit_length=20
```

```
x=800884c1 bit_length=32
x=8008d103 bit_length=32
x=80086398 bit_length=32
x=0008614d bit_length=20
x=80086095 bit_length=32
x=00073949 bit_length=19
x=8007f623 bit_length=32
x=80065147 bit_length=32
x=800765e6 bit_length=32
x=0006bb27 bit_length=19
x=8005ebe8 bit_length=32
x=80051cbd bit_length=32
x=80053b87 bit_length=32
x=80049249 bit_length=32
x=0004c270 bit_length=19
x=00058a21 bit_length=19
x=000418cb bit_length=19
x=00041021 bit_length=19
x=0003b283 bit_length=18
x=00039766 bit_length=18
x=80037c8c bit_length=32
x=0003cd44 bit_length=18
x=00034919 bit_length=18
x=80029cd5 bit_length=32
x=0002169f bit_length=18
x=0001be89 bit_length=17
x=8001e072 bit_length=32
x=8000b17b bit_length=32
x=00010ec5 bit_length=17
x=8000900e bit_length=32
x=000014ac bit_length=13
x=40000049 bit_length=31
```

The first column shows N_i for $i \in [0, 31]$, and the second column shows N_i for $i \in [32, 63]$. We can see that no values in the first column are greater than 20-bits, this is expected as they only involve 8-bit factors.

2. Is there a bit shift $p \ll i$ such that $f_{2^i p}(x)$ has small coefficients? This is the key trick needed to turn arbitrary short-sleeve values into polynomials with small coefficients.

Consider the last limb of p, p_0. Assume, WLOG, that

$$p_0 = \text{0x80000067} = \text{0b10000000000000000000000001100111}$$
$$\Rightarrow p_0 \ll 1 = \text{0x1000000ce} = \text{0b10000000000000000000000011001110}$$

Observe that $p_0 \ll 1$ overflows the 32-bit limb; in effect this turns the polynomial into a monic, with the second coefficient being $p_0 \ll 1 \bmod 2^{32} = \text{0xce}$. And because the rest of the coefficients of $f_p(x)$ are $x \in [\text{0x01}, \text{0xff}]$, $2x \in [\text{0x02}, \text{0x1fe}]$. So we know that $f_{2p}(x)$ has small coefficients, hence the bit shift we are looking for is $i = 1$.

3. If $f_{2^i p}(x)$ and $f_{2^j q}(x)$ have small coefficients, can you still compute $f_{2^i p}(x) * f_{2^j q}(x)$ from public information? Can you still recover p and q?

We know $i = 1$ (from q2). Because $f_{2p}(x)$ represents $2p$ as a polynomial, $f_{2p}(x) * f_{2q}(x)$ represents $2p \times 2q = 4n$ as a polynomial.

So we can factor f_{4n}, and halve the resulting values to recover p and q.

4. If this polynomial factorization technique worked for every p and q, then RSA would be broken. Why is the short-sleeve property important, and why doesn't this factorization method work in general? What are the limits?

Note that integer polynomial factorization (that is, factorization over $\mathbb{Z}$) depends on two factors (no pun intended): coefficient size and degree of polynomial.

For short-sleeve RSA moduli, we exploited the fact that the largely uncovered limbs gave rise to small coefficients, using our suitable value of limb size. Hence, we can construct a polynomial representation of the integer with relatively small coefficients and degree so that polynomial factorization is "fast". Indeed, the short-sleeve property is important.

For uncorrupted RSA moduli, we do not have such small coefficient property. As an *a priori* bound for estimating the complexity of factorization algorithms, we can consider the Landau-Mignotte bound, plagarized from *Landau-Mignotte bound* by Wikipedia:

Let $f(x), h(x) \in \mathbb{Z}[x]$, where $h(x)$ divides $f(x)$. Denote the sum of the absolute values of the coefficients in $h(x)$ and $f(x)$ as $\|h\|_1$ and $\|f\|_1$, respectively. We then have

$$\|h\|_1 \leq 2^n \|f\|_1$$

where $n = \deg f(x)$.

Thus, if $\|f\|_1$ is large, then we have a large bound for $\|h\|_1$, which means we have a large coefficient space to traverse through. As far as I know, most algorithms that factorize univariate integer polynomials require a specified bound for the coefficient of the factors (link: *Factoring univariate polynomials over the integers*), so the complexity of factorization is dependent on this bound.

We use the Landau-Mignotte bound to informally justify why uncorrupted RSA moduli always produce a large bound, hence a "slow" factorization, with two cases below:

1. We choose a large limb for a low degree polynomial. Then the coefficients of $f_n(x)$ are very large, so $\|f\|_1$ is large, which yields a large coefficient space for its factors.
2. We choose a small limb for small coefficients. Then the degree of $f_n(x)$ is large, so the 2^n factor in the inequality dominates, which yields a large coefficient space for its factors.

In both cases, we have a large coefficient space for the factors of $f_n(x)$, hence why factorization is infeasible for uncorrupted RSA.

5. The short-sleeve property allows us to construct the product $f_{2^i p}(x) * f_{2^j q}(x)$, but unless $f_{2^i p}(x)$ and $f_{2^j q}(x)$ are irreducible, factorization may split this into more than two terms. Prove that there is always an efficient way to recover p and q from the polynomial factorization.

Note that `short-sleeve_rsa2.py` gives code on how to do this. We describe the process mathematically below:

Suppose $f_{2^i p}(x)\, f_{2^i q}(x)$ factorizes into $g_1(x) \cdots g_k(x)$ for integer $k \geq 2$. Note that for the chosen base $B = 2^{32}$, we have

$$f_{2^i p}(B)\, f_{2^i q}(B) = g_1(B) \cdots g_k(B) = 2^{i+j} pq$$

Thus, exactly two of the factors $g_i(B)$ contains p or q, while other factors are powers of 2.
Therefore, we can traverse through all the factors after factorization and check if the factor is of the form 2^k (a power of 2) or $2^k p$ for prime p, which is efficient. Then we can extract p and q easily.

□